CS 185/285 Assignment 4: RL for LLMs (GR-REINFORCE and GRPO) — Report

Setup note. All runs were executed locally on NVIDIA RTX A6000 GPUs (48 GB) instead of Modal, with wandb in offline mode. The hyperparameters are exactly those of the handout commands; the only differences are that `--output_dir` is a local path (`runs/<name>` instead of `/vol/runs/modal_<name>`) and that the submission bundle was built with the local entry point `python -m hw4.gradescope_bundle`. The launcher is `scripts/run_local.sh`, and the figures are produced by `scripts/plot_report.py` from each run's `metrics.jsonl`. Every configuration was run once (seed 0), so small differences between runs may be noise.

Results of the four required runs

Run	Final train reward	Final eval exact match	Handout expectation	Met
format_copy + GRPO (51 iterations)	1.29 (mean of last 10)	1.00	reward about 1.3, eval near 1.0	yes
format_copy + GR-REINFORCE (51 iterations)	1.30 (mean of last 10)	1.00	reward about 1.3, eval near 1.0	yes
math_hard + GR-REINFORCE (201 iterations)	0.29 (mean of last 20)	0.287	train about 0.32, eval about 0.31 (as low as 0.29); debug below 0.27	close (eval 0.003 below the stated range)
math_hard + GRPO (501 iterations)	0.56 (mean of last 20)	0.400	train above 0.55, eval above 0.37	yes

The eval metric is the boxed-answer exact match on the fixed 512-problem held-out subset. Before training it is 0.223 for both math runs (handout: about 0.23). The math runs took 4.1 hours (GR-REINFORCE) and 10.1 hours (GRPO) of training time on one A6000 each.

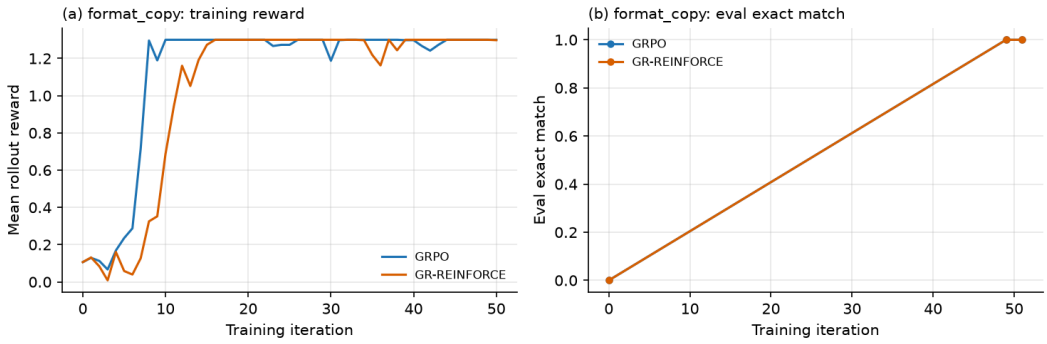


Figure 1: format_copy

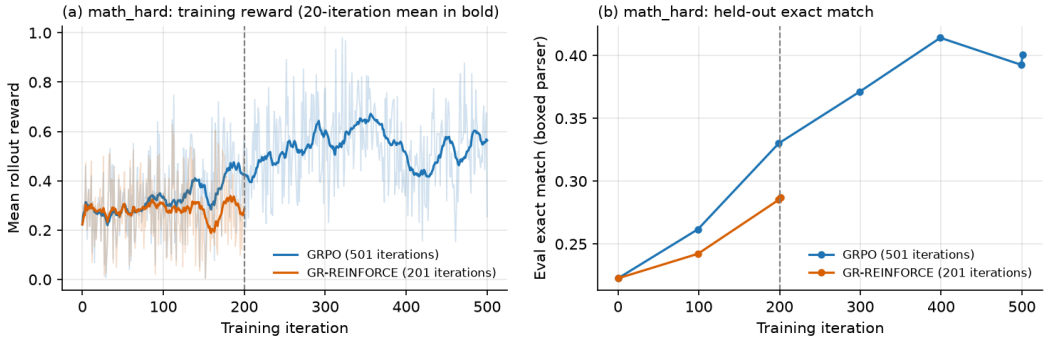


Figure 2: math_hard

1. Approximate KL

Let $\Delta = \log \pi_{\text{ref}}(a) - \log \pi_\theta(a)$ be the log-ratio at a token a that was **sampled from the current policy** π_θ. Then

$$\mathbb{E}_{a\sim\pi_\theta}\left[e^{\Delta} - \Delta - 1\right] = \sum_a \pi_\theta(a)\,\frac{\pi_{\text{ref}}(a)}{\pi_\theta(a)} - \mathbb{E}_{a\sim\pi_\theta}[\Delta] - 1 = 1 + \mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}}) - 1 = \mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}}).$$

So the estimator is unbiased for the KL between the current policy and the frozen reference model at that position (exactly so when the token was sampled from the current policy, i.e. on the first pass over a rollout, and approximately after the policy has moved within an iteration), using only the log-probabilities of the one token that was actually generated. The term $e^{\Delta} - 1$ has zero mean and acts as a control variate: unlike the plain estimator $-\Delta$, the quantity $e^{\Delta} - \Delta - 1$ is non-negative for every sample (since $e^x \geq 1 + x$), equals zero exactly when the two policies agree on the token, and behaves like $\Delta^2/2$ for small Δ, so it has lower variance and can never produce a negative "KL".

The exact KL needs the full next-token distributions of both models at every position: $\sum_v \pi_\theta(v)\,(\log \pi_\theta(v) - \log \pi_{\text{ref}}(v))$ over a vocabulary of about 151K tokens. That means materializing two (batch × length × vocabulary) log-probability tensors and keeping the current policy's one in the autograd graph. For one minibatch of 8 sequences of about 700 tokens this is 8 × 700 × 151K ≈ 0.85 billion entries per tensor (over 3 GB in float32 each, before gradients), whereas the sampled estimator needs one gathered scalar per token, which the policy-gradient loss computes anyway, and the reference log-probabilities are already stored in the rollout buffer.

2. Implementation

Order of implementation (the order suggested by the handout, bottom-up so that each piece could be checked before the next one used it):

1. `hw4/models/logprobs.py`: `compute_per_token_logprobs`, `build_completion_mask`, `approx_kl_from_logprobs`.
2. `hw4/rollout/rollout_buffer.py`: `iter_minibatches`.
3. `hw4/train.py`: `compute_group_advantages`, `maybe_normalize_advantages`.
4. `hw4/rl/reinforce.py`: the GR-REINFORCE update.
5. `hw4/rl/grpo.py`: the GRPO update (clipped ratio, sampled-token KL penalty, clip fraction).

Then format_copy with both algorithms as an end-to-end check, and only then the math runs.

Confusion point that had to be resolved: memory of the no-grad log-probability pass. On math_hard a rollout is 64 sequences of roughly 160 prompt tokens plus up to 512 generated tokens. Computing the old-policy log-probabilities for the whole rollout in one forward pass creates a (64 × length × 151K) logits tensor, which does not fit in 48 GB even without gradients. I resolved it by processing the rows in chunks of 8 in the no-grad path and by computing the cross-entropy on float32 logits (the model runs in bfloat16, and log-softmax over 151K classes in half precision loses accuracy for low-probability tokens). Even so, `nvidia-smi` showed about 48.5 GB of the card's 49 GB in use during the GRPO math run, so the chunk size is the first thing to lower on a smaller GPU. A second, smaller point was the group advantage: a group in which all completions receive the same reward has zero standard deviation, so its advantages are set to zero instead of dividing by a near-zero number; such groups then contribute no policy gradient (on math_hard this is the common case early on, when all eight completions of a prompt are wrong).

3. GR-REINFORCE vs. GRPO on math_hard

Over the first 200 iterations (the part where both runs exist):

Window (iterations)	Metric	GR-REINFORCE	GRPO
0–19	mean train reward	0.277	0.266
80–99	mean train reward	0.306	0.331
180–199	mean train reward	0.269	0.423
99	eval exact match	0.242	0.262
199	eval exact match	0.285	0.330
180–199	completions containing a boxed answer	0.369	0.527
180–199	completions cut off at 512 tokens	0.640	0.484
180–199	approximate KL to the reference	0.0021	0.0092

The two runs are indistinguishable for the first 50 iterations or so, then GRPO pulls ahead steadily: at iteration 199 its held-out exact match is 0.330 against 0.285, and its training reward is about 0.15 higher. GR-REINFORCE improves the eval metric (0.223 ⟶ 0.287) but its noisy training reward shows no clear trend within 200 iterations.

Why the comparison is interesting. The provided commands give both methods the same learning rate, the same rollout (8 prompts × 8 completions per iteration), the same minibatch size and the same gradient accumulation (8 minibatches ⟶ one optimizer step per pass over the rollout). The only difference is that GRPO makes two passes (`ppo_epochs = 2`) over each rollout and GR-REINFORCE makes one. Because generation dominates the cost, the wall-clock time per iteration is practically identical (72.4 s for GRPO versus 73.8 s for GR-REINFORCE). So at equal sampling cost and equal wall-clock time, GRPO takes twice as many optimizer steps, and the comparison isolates what the PPO-style machinery buys: the importance ratio makes the second pass over the same samples a valid (off-policy corrected) update, and the clipping plus KL penalty keep that reuse from moving the policy too far. The KL column shows GRPO has indeed moved about four times further from the reference policy by iteration 200, without becoming unstable.

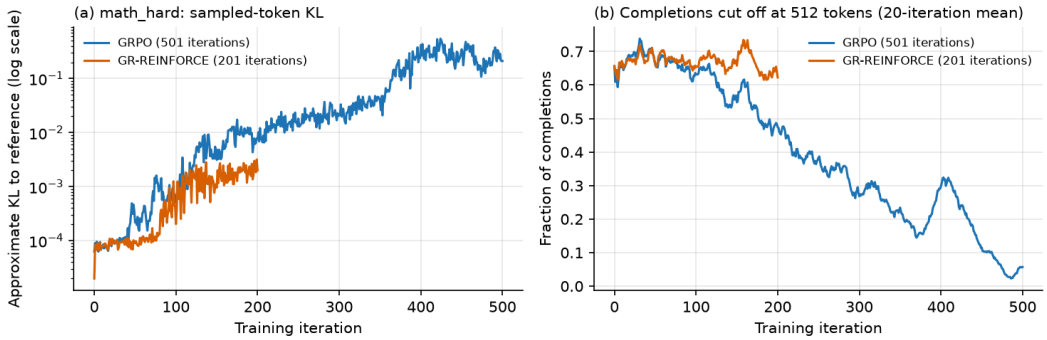


Figure 3: math_hard diagnostics

What GRPO learned after iteration 200. The eval exact match keeps rising to 0.414 at iteration 399 and ends at 0.400. Much of the gain comes from learning to finish: the fraction of completions that hit the 512-token limit falls from 0.68 at the start to 0.06 at the end, the mean completion length falls from 473 to 298 tokens, and the fraction of completions that contain `\boxed{...}` rises from 0.32 to 0.94. A completion that never reaches its final answer gets zero reward regardless of how good the reasoning was, so shortening the solution is the cheapest way to increase the reward. In the last 100 iterations the run shows signs of drifting: the KL to the reference rises to about 0.24, the mean token entropy rises from about 0.1 to above 1, the training reward dips between iterations 400 and 450, and the eval metric stops improving (0.414 ⟶ 0.393 ⟶ 0.400). I would not expect further training with these settings to help without a larger KL coefficient or a lower learning rate.

4. GRPO ablations on format_copy

Each run changes one thing relative to the default command (`ppo_epochs 2, minibatch_size 8, grad_accum_steps 6, kl_coef 0.05, clip_eps 0.2`), with the same seed. A rollout is 8 × 6 = 48 completions. "First ≥ 1.2" is the first iteration whose rollout reward is at least 1.2 (the maximum is 1.3).

Run	Change	Optimizer steps / iteration	First ≥ 1.2	Reward, last 10	Min reward, last 20	Final eval	KL, last 10	Mean clip fraction
default	—	2	8	1.29	1.24	1.00	0.26	0.002
`abl_epochs1`	ppo_epochs 1	1	14	1.30	1.25	1.00	0.25	0.000
`abl_epochs4`	ppo_epochs 4	4	8	1.28	1.16	1.00	0.26	0.010
`abl_kl0`	kl_coef 0	2	8	1.30	1.30	1.00	0.28	0.002
`abl_kl1`	kl_coef 1	2	never	0.77	0.56	1.00	0.11	0.002
`abl_kl10`	kl_coef 10	2	never	0.12	0.03	0.00	0.001	0.002
`abl_clip0.05`	clip_eps 0.05	2	8	1.29	1.24	1.00	0.25	0.031
`abl_clip0.5`	clip_eps 0.5	2	8	1.29	1.24	1.00	0.24	0.000
`abl_accum1`	grad_accum_steps 1	12	6	1.12	0.14	1.00	0.51	0.044
`abl_mb16_accum3`	minibatch 16, accum 3	2	8	1.29	1.24	1.00	0.23	0.003
`abl_mb48_accum1`	minibatch 48, accum 1	2	8	1.28	1.21	1.00	0.24	0.002

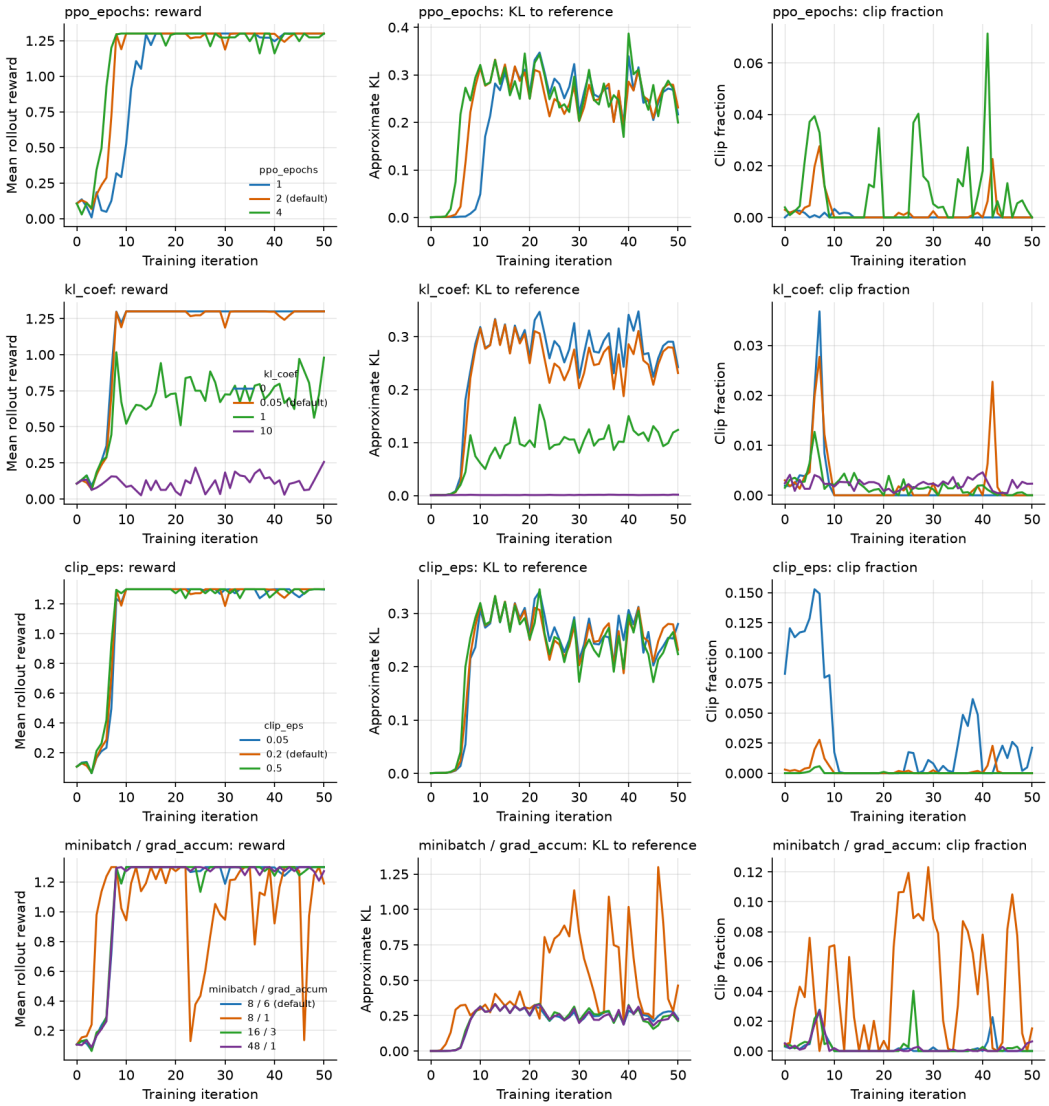


Figure 4: GRPO ablations on format_copy

Which hyperparameters mattered most. The KL coefficient and the number of optimizer steps per rollout (set by `grad_accum_steps` together with `ppo_epochs`). The clipping threshold and the minibatch size at a fixed number of optimizer steps had almost no effect on this task.

- **`kl_coef`.** This had the largest effect. With 10, the penalty dominates the loss: the KL stays at about 0.001, i.e. the policy never leaves the reference model, the reward stays at the initial 0.1 and the final eval is 0.00. With 1, the policy does learn, but the penalty holds the KL near 0.11 while the solved task needs about 0.25–0.3 nats per token of divergence from the reference; the sampling policy therefore stays partly mixed (reward 0.5–1.0), although greedy evaluation is already 1.00. With 0, the result is as good as the default (and marginally steadier in the last 20 iterations): on a task this easy the clipping and the small learning rate are enough of a trust region, and the default 0.05 is effectively inactive.
- **`ppo_epochs`.** One epoch is in effect one policy-gradient step per rollout: it reaches the same final reward but needs 14 iterations instead of 8 to get there, and its clip fraction is zero because the ratio is exactly 1 on the first pass. Four epochs learn about as fast as two (the first rewards above 1.0 arrive one iteration earlier), and the clip fraction becomes non-zero repeatedly (peaks up to 0.07) with a slightly lower minimum reward late in training. Two epochs is the sweet spot here.
- **`grad_accum_steps = 1`.** This takes 12 optimizer steps per rollout instead of 2, each from a single minibatch of 8 completions. It is the fastest to learn (reward ≥ 1.2 at iteration 6) and the only unstable configuration: after iteration 20 the KL jumps to 0.5–1.3, the clip fraction is around 0.1 for long stretches, and the reward repeatedly collapses (down to 0.14) before recovering. Each small-batch step is noisy, the advantage of a group

is spread across different minibatches, and twelve consecutive steps on the same 48 samples let the policy drift far from the policy that generated them, which is exactly the regime where the surrogate objective stops being trustworthy. The greedy eval is still 1.00 at the end, but the training curve is clearly worse.

- **`minibatch_size` in tandem with `grad_accum_steps`.** Keeping the number of optimizer steps at 2 per iteration (16/3 and 48/1) reproduces the default curve almost exactly. This is expected: accumulating 6 minibatches of 8 gives the same gradient as one minibatch of 48, so these settings change memory use, not the optimization. The comparison with `abl_accum1` shows that what matters is the number and size of the optimizer steps, not how the forward passes are chunked.
- **`clip_eps`.** 0.5 never clips (clip fraction 0.000) and 0.05 clips up to 15% of the tokens during the fast-learning phase (iterations 0–9), yet the reward curves are nearly identical. With only two epochs and one optimizer step per epoch, the ratio rarely moves far enough from 1 for the clipping to bind for long.

Settings that made learning worse: `kl_coef` 10 (no learning), `kl_coef` 1 (stuck at a mixed policy), and `grad_accum_steps` 1 (fast but unstable). None of the variants is clearly better than the default; `kl_coef` 0 is competitive with it and was the only run with a reward of 1.30 in each of the last 20 iterations.

5. Qualitative behavior (math_hard, GRPO run)

Example 1 (iteration 249): finishing matters more than reasoning. Problem: *Let* $a \equiv (3^{-1} + 5^{-1} + 7^{-1})^{-1} \pmod{11}$. *What is the remainder when* a *is divided by 11?* (answer: 10). The eight sampled completions for this prompt:

Candidate	Tokens	Reward	Advantage	End of completion
0	433	1.10	+1.73	"...The remainder when a is divided by 11 is `\boxed{10}`."
7	417	1.10	+1.73	"...The remainder when a is divided by 11 is `\boxed{10}`."
4	512 (cut off)	0.10	−0.40	"...we find that $10 \times 10 = 100 \equiv 1 \pmod{11}$. So, 10^{-1}"
2	512 (cut off)	0.00	−0.61	"...Thus, $a \equiv 10 \pmod{11}$, and the remainder when"
3	512 (cut off)	0.00	−0.61	"...Therefore, $a \equiv 10 \pmod{11}$, and the remainder when a is divided by 11 is"
1, 5, 6	512 (cut off)	0.00	−0.61	all three had derived $a \equiv 10 \pmod{11}$ but had not boxed it

I found this informative because every one of the eight completions computes the right answer, but six of them spend the token budget on a step-by-step search for modular inverses and are cut off, in one case immediately before the boxed answer. Only the two shorter completions are rewarded. The group-relative advantage therefore pushes the policy towards the more concise derivations, not towards "better mathematics". This matches the aggregate statistics in Section 3 (truncation falling from 68% to 6%) and explains a large part of the gain in exact match. It also shows how much the measured "accuracy" of the base model (0.223) under-states what it can solve when it is given room to finish.

Example 2 (iteration 499): a group with no learning signal. Problem: *Let* x *and* y *be real numbers greater than 1 such that* $(\log_2 x)^4 + (\log_3 y)^4 + 8 = 8(\log_2 x)(\log_3 y)$. *Compute* $x^{\sqrt{2}} + y^{\sqrt{2}}$. (answer: 13). All eight completions solve it in 395–457 tokens starting from a substitution for the two logarithms, and end in `\boxed{13}`; all rewards are 1.10 and all advantages are 0. The completions are nearly paraphrases of each other. Groups like this (all correct), like the opposite case (all wrong), carry no gradient. The fraction of completions

with a non-zero advantage is 0.52 over the first 50 iterations, peaks at about 0.70 around iterations 200–250 and falls back to 0.60 over the last 50, so a large share of every rollout is uninformative throughout training. By contrast, at iteration 9 a sampled group (an inscribed-squares geometry problem with answer 251) had all eight completions cut off at 512 tokens with reward 0, again giving zero advantage for the whole group.

Commands

```
# Four required runs (second argument: GPU index)
scripts/run_local.sh format_copy_grpo 0
scripts/run_local.sh format_copy_reinforce 0
scripts/run_local.sh math_hard_reinforce 0
scripts/run_local.sh math_hard_grpo 1

# GRPO ablations on format_copy
for name in abl_epochs1 abl_epochs4 abl_kl0 abl_kl1 abl_kl10 abl_clip0.05 abl_clip0.5 \
            abl_accum1 abl_mb16_accum3 abl_mb48_accum1; do
  scripts/run_local.sh $name 0
done

# Figures and submission bundle
uv run --with matplotlib python scripts/plot_report.py
uv run python -m hw4.gradescope_bundle \
  --run_dir runs/format_copy_grpo --run_dir runs/format_copy_reinforce \
  --run_dir runs/math_hard_grpo --run_dir runs/math_hard_reinforce \
  --output_dir ../hw4_gradescope_submission --overwrite
```